27 FEB 2026

AADK TASK 1

State Concepts Deep Dive & Visual Mapping

MindMatrix

1. Key Terms

State

State is the data that controls what is shown on the screen.
When the state changes, the UI automatically updates to show the new value.

Example:

```
var count = 0
Button(onClick = { count++ }) {
   Text("Count: $count")
}
```

When count changes, the text updates.

remember()

remember() is used to store a value inside a composable so that it does not reset every time the screen recomposes. It “remembers” the value while the composable is alive.

Example:

```
var count = remember { mutableStateOf(0) }
```

mutableStateOf()

mutableStateOf() creates a state object that can change. When its value changes, Compose recomposes the UI that depends on it.

Example:

```
var name by remember { mutableStateOf("") }
```

If name changes, the UI using it will update.

State Hoisting

State hoisting means moving the state to a higher-level composable instead of keeping it inside a child composable.
This makes the child composable reusable and easier to manage.

Example:

```
@Composable
fun Parent() {
    var text by remember { mutableStateOf("") }
    Child(text, onTextChange = { text = it })
}

@Composable
fun Child(value: String, onTextChange: (String) -> Unit) {
    TextField(value = value, onValueChange = onTextChange)
}
```

Here, the state is in Parent, not inside Child.

Recomposition

Recomposition happens when Compose redraws part of the UI because the state has changed. It only updates the parts that depend on the changed state.

Example:
If a Switch changes from false to true, the UI parts using that value will recompose.

Reflection

One challenge is managing state properly, especially when multiple components depend on the same data. Another challenge is avoiding unnecessary recomposition that can affect performance. Handling edge cases (like rapid clicks or invalid states) can also be difficult. I plan to address these by learning proper state management (ViewModel, state hoisting), testing UI behavior thoroughly, and following clean UI architecture principles.

Outcome Understanding

From this task, I understood the full flow clearly:

User taps button → onClick handler runs → state updates → Compose recomposes affected UI → updated result is displayed.

This helped me understand how interactive UI works internally in Jetpack Compose.

2. Interaction Scenario

Scenario Chosen: Tip Calculator

User enters:

- Bill amount
- Toggles “Round up tip” switch

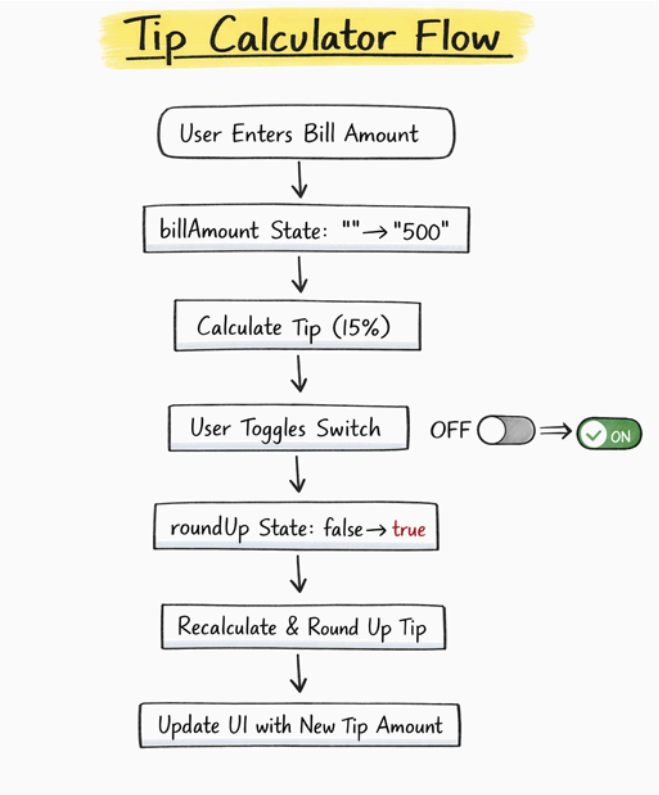
App:

- Calculates 15% tip
- If switch is ON → round up tip
- Displays final tip amount

State Variables

State Variable	Initial Value	When It Changes
billAmount	""	When user types
roundUp	FALSE	When switch toggled
tipAmount	0	When billAmount or roundUp changes

Flow Diagram



3. State-Handling Strategy Table

UI Component	State Variable	Stateful or Stateless?	Where is State Defined?	Comments
TextField (Bill Amount)	billAmount	Stateless	Parent Composable	Gets value and onValueChange from parent
Switch (Round Up)	roundUp	Stateless	Parent Composable	Only shows state and triggers event
Text (Tip Result)	tipAmount	Stateless	Calculated in Parent	Displays calculated result
Parent Screen	billAmount, roundUp	Stateful	Top-level composable	Controls whole UI logic

4. Reflection

State hoisting is important because it makes composables reusable and easier to manage. If state is kept deep inside child composables, it becomes hard to control or share that state with other components. It can also make debugging difficult.

If state is not managed properly, the UI might not update correctly. For example, if the round-up switch keeps its own internal state and the parent doesn't know about it, the tip might not recalculate correctly when toggled. This can cause bugs where the switch changes but the result stays the same.